

Solution Requirements

Date	01/07/2026
Team ID	
Project Name	Smart Lender
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

The requirements below define exactly what the Smart Lender loan-approval prediction system will do and how well it will perform, based on the current Flask application, trained Random Forest model, and dataset.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The current release does not implement user login. The loan-prediction form (/predict) and result page are openly accessible to any bank staff member. (Recommended for a future release: staff login via username/password before accessing the prediction tool.)	Low
2	Authorization levels	No role-based access control is implemented in the current version - there is a single implicit user type (bank/loan officer) with full access to the Home, Predict, and Result pages. Future versions could add Admin vs. Officer roles.	Low
3	External interfaces	No third-party APIs or external databases are used. The system loads two local files at startup - the trained model (rdf.pkl) and the fitted feature scaler (scale1.pkl) - both read via Python pickle within the Flask app.	Medium
4	Transactions processing	Each loan application is a single, stateless transaction: the applicant's details are submitted via an HTTP POST request to /submit, scaled with scale1.pkl, and passed to the Random Forest model (rdf.pkl) for an immediate prediction. No payment processing is involved.	High
5	Reporting	The system generates a single real-time result - "Loan Will be Approved" or "Loan Will Not be Approved" - displayed on the result page (submit.html) immediately after submission. No dashboards, historical reports, or data exports exist in the current scope.	Medium
6	Business rules, etc.	Loan eligibility is determined entirely by the trained ML model using 11 input features (Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area). If the model output is 1, the loan is marked approved; if 0, it is marked not approved.	High
7	Compliance to laws or regulations	No formal compliance framework is implemented yet. Applicant data is processed only in-memory for the current request and is not persisted to a database. For production banking use, compliance with data-protection regulations (e.g., RBI data-localization	Medium

		guidelines, GDPR where applicable) is recommended.	
8	<i>Other</i>	An informational Home page (home.html) introduces the system, its features, and accuracy statistics before directing users to the prediction form. A separate Training notebook (Loan Prediction using ML.ipynb) supports periodic model retraining as new data becomes available.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	A single loan prediction (scaling + Random Forest inference) completes almost instantly since the model and scaler are loaded once at app startup and reused for every request.	Prediction response returned in under 2 seconds per request
2	Scalability	The Flask app is served through gunicorn, a production WSGI server, which can run multiple worker processes and be deployed on a cloud platform (e.g., Render) to handle growth in concurrent users.	Supports at least 50-100 concurrent prediction requests without failure
3	Security & Data Privacy	Applicant data is submitted via a standard HTML form POST and is not persisted to disk or a database, limiting data-at-rest exposure. For production deployment, HTTPS/TLS should be enforced by the hosting platform.	All traffic served over HTTPS (TLS 1.2+); no applicant data stored beyond the request lifecycle
4	Reliability & Availability	The app is designed to run continuously under gunicorn with automatic restart on failure when deployed on a managed cloud host, so the prediction service stays available to bank staff.	Target uptime of 99% per month
5	Usability & Accessibility	The prediction form (predict.html) uses simple labeled dropdowns and number inputs so non-technical bank staff can submit an application in under a minute; result is shown in plain language ("Loan Will be Approved"/"Loan Will Not be Approved").	New users can complete a prediction without training in under 5 minutes
6	<i>Other</i>	Maintainability: the project separates concerns into Dataset, Training (notebook), and Flask (app.py, templates, static, model files), so the model can be retrained and rdf.pkl/scale1.pkl swapped without changing application code.	Model can be retrained and redeployed with no changes to app.py